

```
$ ./build && flash # no vendor library – datasheet only
```

Bare-Metal Peripheral Drivers

UART · I2C · SPI on STM32 Cortex-M, written at register level and verified on the wire.

// OVERVIEW

This project developed bare-metal peripheral drivers in C for the three most widely used embedded communication interfaces — UART, I2C and SPI — on an STM32 Cortex-M microcontroller. Crucially, the drivers were written without any hardware abstraction layer (HAL) or vendor library: all peripheral configuration and data transfer was implemented through direct register-level programming, guided by the device reference manual.

// MOTIVATION & APPROACH

Vendor HALs are convenient but conceal the underlying hardware, limiting both understanding and low-level debugging. Working bare-metal meant reading the reference manual directly: identifying peripheral register maps, enabling clocks through the RCC registers, configuring GPIO alternate functions, and programming each peripheral's control, status and data registers for correct operation.

// DRIVERS IMPLEMENTED

● ● ●

async serial

UART

- › Baud-rate generation from the peripheral clock
- › Frame config: data bits, parity, stop bits
- › Status-flag-driven TX / RX routines

● ● ●

2-wire master

I2C

- › Full master transaction sequence
- › Start, 7-bit addr, R/W, ACK, stop
- › Talks to real external slave ICs on the bus

● ● ●

full-duplex

SPI

- › CPOL / CPHA configured per device
- › Manual chip-select management
- › Byte-level exchange via the data register

Each driver was exercised against real external ICs rather than loopback alone, confirming correct multi-device behaviour on shared buses.

// VERIFICATION & DEBUGGING

Correctness was established at the electrical level, not merely in software. An oscilloscope verified signal integrity, voltage levels and clock timing; a logic analyser captured and decoded complete bus transactions — baud rate and framing on UART, address and acknowledge sequencing on I2C, and clock-phase alignment on SPI. This instrument-led method made it possible to separate firmware faults from wiring or device-configuration faults quickly and decisively.

// OUTCOMES & SKILLS DEMONSTRATED

The project delivered a set of working, reusable register-level drivers and a thorough practical understanding of how embedded communication protocols operate on the wire. It demonstrates proficiency in embedded C, datasheet- and reference-manual-driven development, register-level programming on ARM Cortex-M, serial-protocol theory and practice, and professional debugging with oscilloscopes and logic analysers.